

使用 Web Bluetooth 控制 PLAYBULB 蠟燭

來源：<https://codelabs.developers.google.com/codelabs/candle-bluetooth?hl=zh-tw>

步驟數：8

憑藉新穎的 Web Bluetooth API，不必像 JavaScript 一樣能建立可控制 LED 無燃蠟燭的網頁應用程式。

目錄

1. [這是什麼？](#)
2. [優先播放](#)
3. [做好準備](#)
4. [認識蠟燭](#)
5. [讀些什麼給我聽。](#)
6. [變更顏色](#)
7. [再努力一下](#)
8. [就是這麼簡單！](#)

步驟 1 · 約 1 分鐘

1. 這是什麼？



在本程式碼研究室中，您將瞭解如何透過 [Web Bluetooth API](#)，只使用 JavaScript 控制 [PLAYBULB LED 無焰蠟燭](#)。過程中，您也會使用 JavaScript ES2015 功能，例如「類別」、「箭頭函式」、「對應」和「Promise」。

課程內容

- 如何在 JavaScript 中與附近的藍牙裝置互動
- 如何使用 ES2015 類別、箭頭函式、對應和 Promise

軟硬體需求

- 對網頁開發有基本瞭解
 - [藍牙低功耗](#) (BLE) 和 [通用屬性設定檔](#) (GATT) 的基本知識
 - 您選擇的文字編輯器
 - Mac、Chromebook 或 Android M 裝置，並安裝 [Chrome 瀏覽器應用程式](#)，以及 USB Micro 對 USB 傳輸線。
-

步驟 2 · 約 1 分鐘

2. 優先播放

建議您先前往 <https://googlecodelabs.github.io/candle-bluetooth> 查看即將建立的應用程式最終版本，並使用手邊的 PLAYBULB Candle 藍牙裝置試玩，再開始進行本程式碼研究室。

你也可以觀看我變更顏色的影片：<https://www.youtube.com/watch?v=fBCPA9glxIU>

步驟 3 · 約 1 分鐘

3. 做好準備

下載範例程式碼

您可以下載此處的 ZIP 檔案，取得這項程式碼的程式碼範例：

或複製這個 git 存放區：

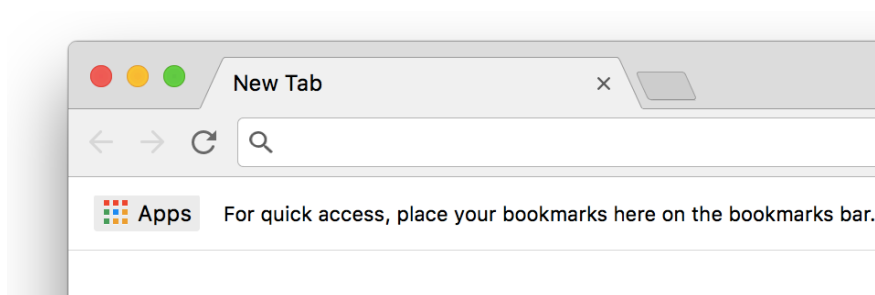
```
git clone https://github.com/googlecodelabs/candle-bluetooth.git
```

如果您下載的來源是 ZIP 檔案，解壓縮後應該會得到根資料夾 `candle-bluetooth-master`。

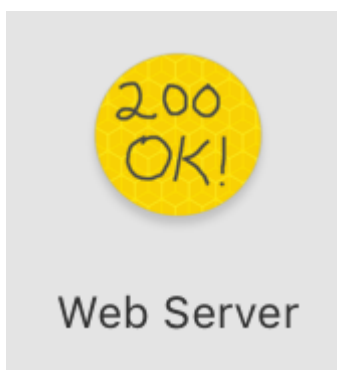
安裝及驗證網路伺服器

您可以自由使用自己的網頁伺服器，但本程式碼研究室的設計可與 Chrome 網頁伺服器完美搭配。如果尚未安裝該應用程式，可以從 Chrome 線上應用程式商店安裝。

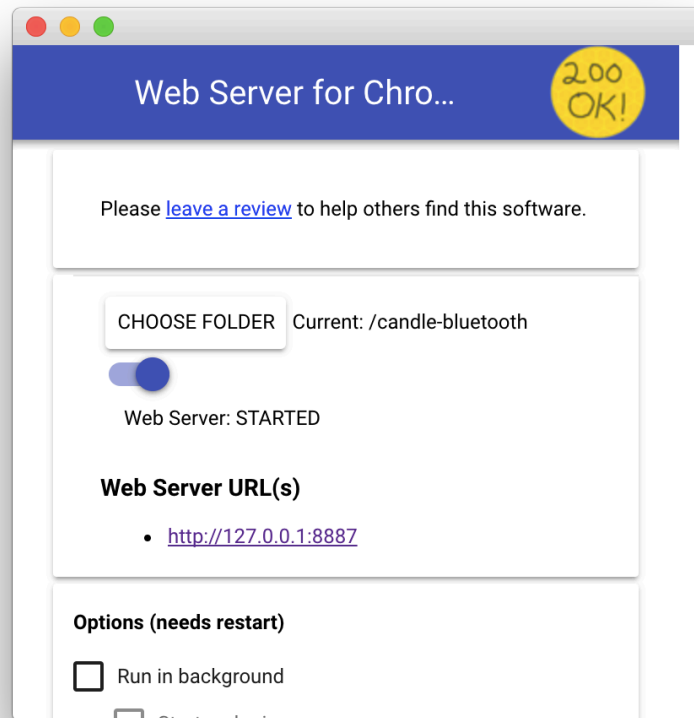
安裝 Chrome 專用網頁伺服器應用程式後，請按一下書籤列中的「應用程式」捷徑：



在隨即顯示的視窗中，按一下「Web Server」圖示：

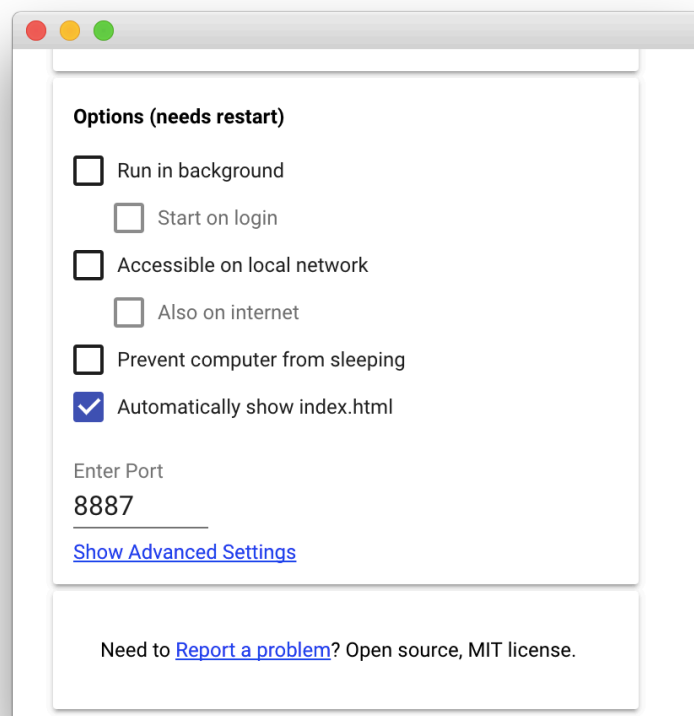


接著會看到這個對話方塊，可供您設定本機網路伺服器：

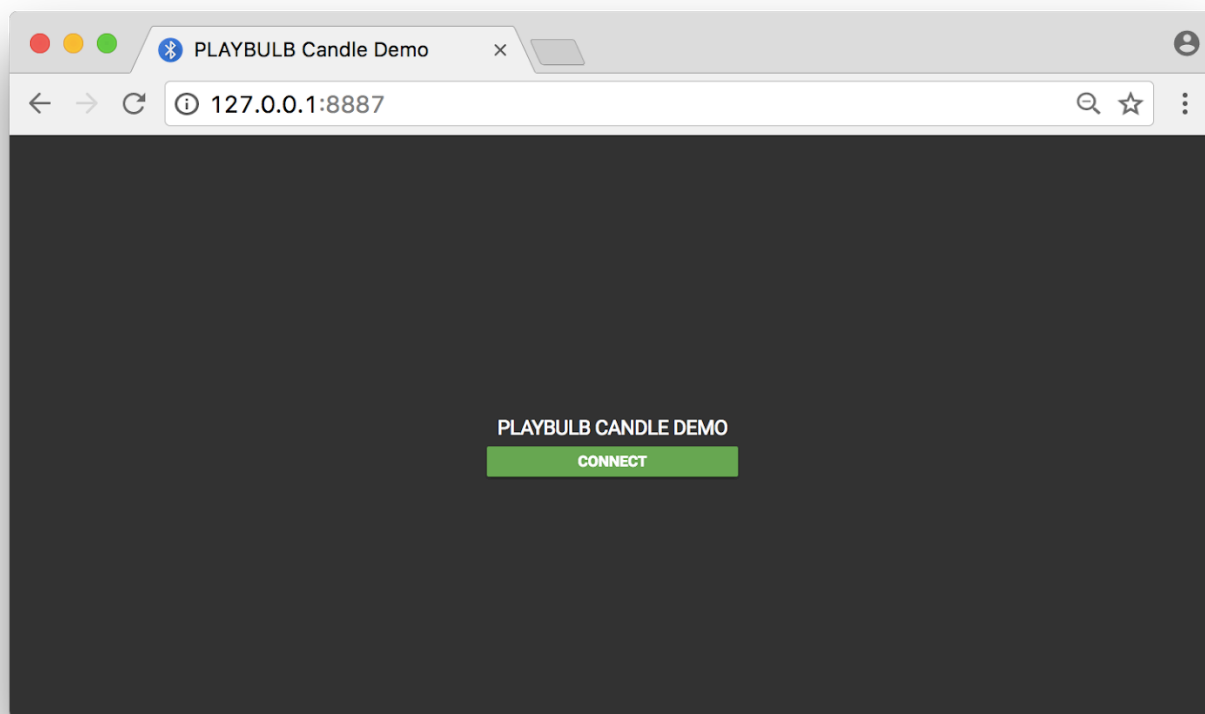


按一下「選擇資料夾」按鈕，然後選取複製 (或解除封存) 的存放區根目錄。這樣您就能透過網頁伺服器對話方塊中醒目顯示的網址 (位於「Web Server URL(s)」部分)，放送進行中的工作。

在「選項」下方，勾選「自動顯示 index.html」旁邊的方塊，如下所示：



現在請在網路瀏覽器中造訪您的網站 (按一下醒目顯示的網路伺服器網址)，您應該會看到類似下方的頁面：



如要查看這個應用程式在 Android 手機上的樣子，您需要啟用「Android 遠端偵錯」，並[設定通訊埠轉送](#) (預設通訊埠號碼為 8887)。完成後，只要在 Android 手機上開啟新的 Chrome 分頁，前往 <http://localhost:8887> 即可。

下一步

目前這個網頁應用程式的功能不多。首先，請新增藍牙支援功能！

4. 認識蠟燭

首先，我們要編寫程式庫，為 PLAYBULB Candle 藍牙裝置使用 JavaScript ES2015 類別。

保持冷靜。類別語法不會為 JavaScript 導入新的物件導向繼承模型。如您所見，這只是提供更清楚的語法，用於建立物件及處理繼承。

首先，在 `playbulbCandle.js` 中定義 `PlaybulbCandle` 類別，並建立 `playbulbCandle` 例項，稍後會在 `app.js` 檔案中使用。

`playbulbCandle.js`

```
(function() {  
  'use strict';  
  
  class PlaybulbCandle {  
    constructor() {  
      this.device = null;  
    }  
  }  
  
  window.playbulbCandle = new PlaybulbCandle();  
  
})();
```

如要要求存取附近的藍牙裝置，我們需要呼叫 `navigator.bluetooth.requestDevice`。由於 PLAYBULB Candle 裝置會持續放送 (如果尚未配對)，因此我們只需定義常數，並將此常數新增至 `PlaybulbCandle` 類別新公開 `connect` 方法中的篩選器服務參數。 `0xFF02`

我們也會在內部追蹤 `BluetoothDevice` 物件，以便日後視需要存取。由於 `navigator.bluetooth.requestDevice` 會傳回 [JavaScript ES2015 Promise](#)，因此我們會在 `then` 方法中執行這項操作。

`playbulbCandle.js`


```

(function() {
  'use strict';

  const CANDLE_SERVICE_UUID = 0xFF02;

  class PlaybulbCandle {
    constructor() {
      this.device = null;
    }
    connect() {
      let options = {filters:[{services:[ CANDLE_SERVICE_UUID ]}]};
      return navigator.bluetooth.requestDevice(options)
        .then(function(device) {
          this.device = device;
        }).bind(this));
    }
  }

  window.playbulbCandle = new PlaybulbCandle();

})();

```

什麼是 Promise ?

Promise 代表值的 Proxy，不一定會在建立 Promise 時得知值。您可以將處理常式與非同步動作的最終成功值或失敗原因建立關聯。這樣一來，非同步方法就能像同步方法一樣傳回值：非同步方法不會傳回最終值，而是傳回日後會取得值的 Promise。

為確保安全性，必須透過使用者手勢 (例如觸控或滑鼠點選) 呼叫 `navigator.bluetooth.requestDevice`，才能探索附近的藍牙裝置。因此，當使用者點選 `app.js` 檔案中的「連線」按鈕時，我們會呼叫 `connect` 方法：

app.js

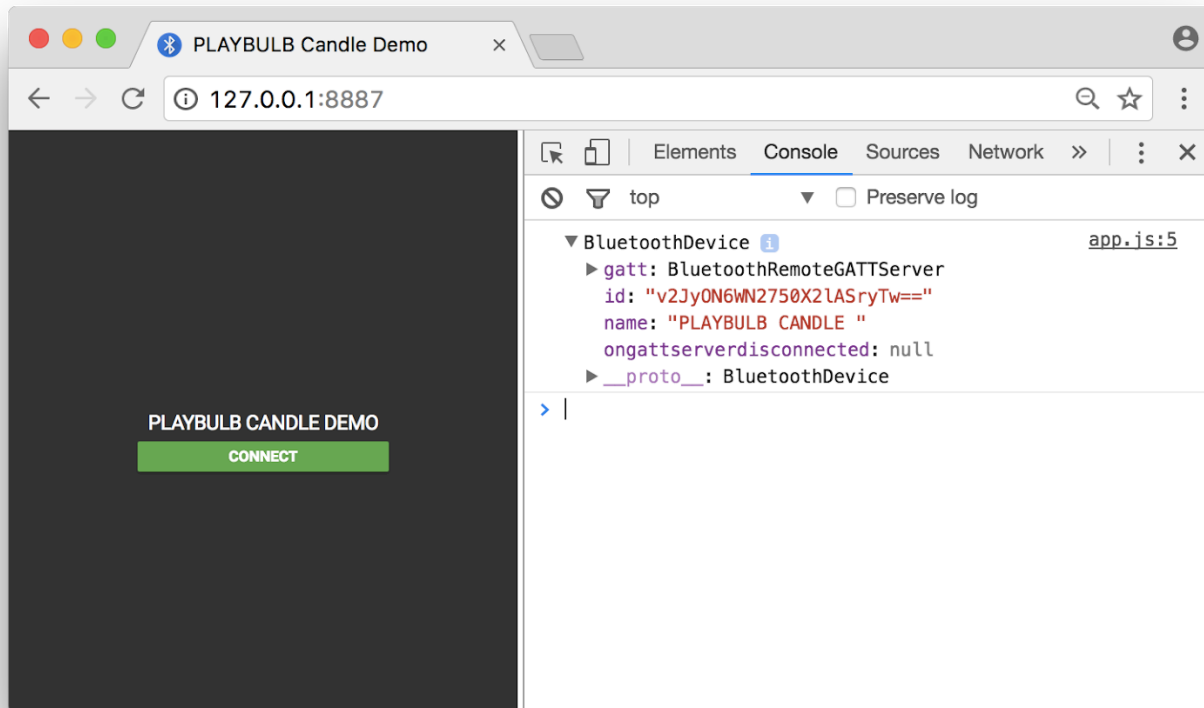
```

document.querySelector('#connect').addEventListener('click', function(event) {
  document.querySelector('#state').classList.add('connecting');
  playbulbCandle.connect()
    .then(function() {
      console.log(playbulbCandle.device);
      document.querySelector('#state').classList.remove('connecting');
      document.querySelector('#state').classList.add('connected');
    })
    .catch(function(error) {
      console.error('Argh!', error);
    });
});

```

執行應用程式

此時，請在網路瀏覽器中造訪您的網站 (按一下網路伺服器應用程式中醒目顯示的網路伺服器網址)，或直接重新整理現有頁面。按一下綠色的「連線」按鈕，在選擇器中挑選裝置，然後使用 `Ctrl + Shift + J` 鍵盤快速鍵開啟您慣用的開發人員工具控制台，並注意記錄的 `BluetoothDevice` 物件。



如果藍牙已關閉和/或 PLAYBULB Candle 藍牙裝置已關閉，你可能會收到錯誤訊息。在這種情況下，請開啟藍牙和裝置，然後再次繼續操作。

請注意，由於 Web Bluetooth API 是新增至 Web 的強大新功能，Google Chrome 的目標是僅允許安全環境使用這項功能。也就是說，您需要考量傳輸層安全標準 (TLS) 來建構。

強制性獎金

我不知道你怎麼想，但我覺得這段程式碼中已經有太多 `function() {}` 了。讓我們改用 `() => {}` JavaScript ES2015 箭頭函式。這類函式絕對是救星：匿名函式的所有優點都具備，但沒有繫結的缺點。

什麼是箭頭函式？

箭頭函式運算式 (又稱粗箭頭函式) 的語法比函式運算式簡短，且會以詞彙方式繫結 `this` 值 (不會繫結自己的 `this`、`arguments`、`super` 或 `new.target`)。箭頭函式一律為匿名。

playbulbCandle.js

```

(function() {
  'use strict';

  const CANDLE_SERVICE_UUID = 0xFF02;

  class PlaybulbCandle {
    constructor() {
      this.device = null;
    }
    connect() {
      let options = {filters:[{services:[ CANDLE_SERVICE_UUID ]}]};
      return navigator.bluetooth.requestDevice(options)
        .then(device => {
          this.device = device;
        });
    }
  }

  window.playbulbCandle = new PlaybulbCandle();

})();

```

app.js

```

document.querySelector('#connect').addEventListener('click', event => {
  playbulbCandle.connect()
    .then(() => {
      console.log(playbulbCandle.device);
      document.querySelector('#state').classList.remove('connecting');
      document.querySelector('#state').classList.add('connected');
    })
    .catch(error => {
      console.error('Argh!', error);
    });
});

```

下一步

- 好... 我真的可以跟這根蠟燭對話嗎？
- 當然，請跳至下一個步驟

常見問題

- [我可以在 JavaScript ES2015 類別中定義靜態方法嗎？](#)
- [支援哪些藍牙 GATT 服務 UUID？](#)
- [哪裡可以找到正式支援的藍牙 GATT 服務清單？](#)
- [藍牙裝置屬性有哪些？](#)

5. 讀些什麼給我聽。

現在您已從 `navigator.bluetooth.requestDevice` 的 Promise 傳回 `BluetoothDevice`，接下來該怎麼做？讓我們呼叫 `device.gatt.connect()`，連線至藍牙遙控器 GATT 伺服器，該伺服器會保留藍牙服務和特徵定義：

playbulbCandle.js

```
class PlaybulbCandle {
  constructor() {
    this.device = null;
  }
  connect() {
    let options = {filters:[{services:[ CANDLE_SERVICE_UUID ]}]};
    return navigator.bluetooth.requestDevice(options)
      .then(device => {
        this.device = device;
        return device.gatt.connect();
      });
  }
}
```

朗讀裝置名稱

我們已連線至 PLAYBULB Candle 藍牙裝置的 GATT 伺服器。現在我們要取得主要 GATT 服務 (先前宣傳為 `0xFF02`)，並讀取屬於這項服務的裝置名稱特徵 (`0xFFFF`)。只要在 `PlaybulbCandle` 類別中新增 `getDeviceName` 方法，並使用 `device.gatt.getPrimaryService` 和 `service.getCharacteristic`，即可輕鬆達成這個目標。 `characteristic.readValue` 方法實際上會傳回 `DataView`，我們只需使用 `TextDecoder` 解碼即可。

playbulbCandle.js

```
const CANDLE_DEVICE_NAME_UUID = 0xFFFF;

...

getDeviceName() {
  return this.device.gatt.getPrimaryService(CANDLE_SERVICE_UUID)
    .then(service => service.getCharacteristic(CANDLE_DEVICE_NAME_UUID))
    .then(characteristic => characteristic.readValue())
    .then(data => {
      let decoder = new TextDecoder('utf-8');
      return decoder.decode(data);
    });
}
```

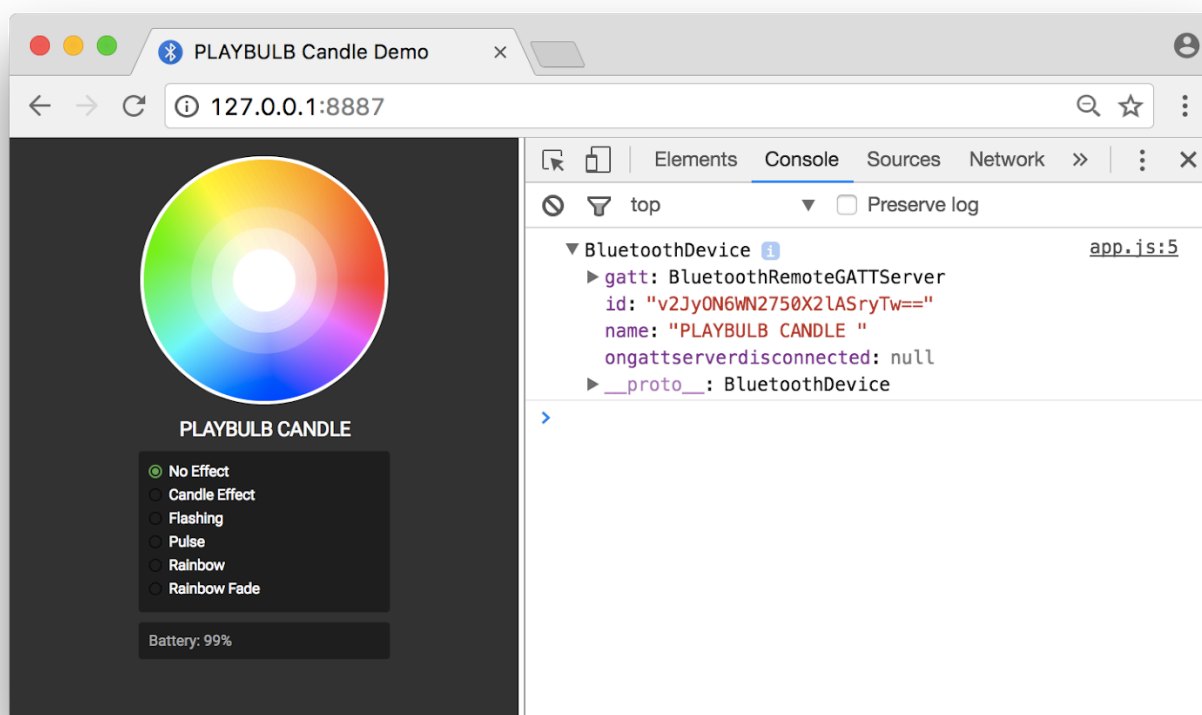
連線後，讓我們呼叫 `playbulbCandle.getDeviceName`，將此項目新增至 `app.js`，並顯示裝置名稱。

app.js

```
document.querySelector('#connect').addEventListener('click', event => {
  playbulbCandle.connect()
  .then(() => {
    console.log(playbulbCandle.device);
    document.querySelector('#state').classList.remove('connecting');
    document.querySelector('#state').classList.add('connected');
    return playbulbCandle.getDeviceName().then(handleDeviceName);
  })
  .catch(error => {
    console.error('Argh!', error);
  });
});

function handleDeviceName(deviceName) {
  document.querySelector('#deviceName').value = deviceName;
}
```

此時，請在網路瀏覽器中造訪您的網站 (按一下網路伺服器應用程式中醒目顯示的網路伺服器網址)，或直接重新整理現有頁面。確認 PLAYBULB Candle 已開啟，然後按一下頁面上的「連線」按鈕，色彩選擇器下方應會顯示裝置名稱。



讀取電池電量

PLAYBULB Candle 藍牙裝置也提供標準電池電量藍牙特徵，可顯示裝置的電池電量。也就是說，我們可以針對藍牙 GATT 服務 UUID 使用 `battery_service` 等標準名稱，針對藍牙 GATT 特徵 UUID 使用 `battery_level`。

讓我們在 `PlaybulbCandle` 類別中新增 `getBatteryLevel` 方法，並以百分比讀取電池電量。

playbulbCandle.js

```
getBatteryLevel() {
  return this.device.gatt.getPrimaryService('battery_service')
    .then(service => service.getCharacteristic('battery_level'))
    .then(characteristic => characteristic.readValue())
    .then(data => data.getUint8(0));
}
```

我們也需要更新 `options` JavaScript 物件，將電池服務納入 `optionalServices` 鍵，因為 PLAYBULB Candle 藍牙裝置不會宣傳這項服務，但存取這項服務仍是必要條件。

playbulbCandle.js

```
let options = {filters:[{services:[ CANDLE_SERVICE_UUID ]}],
               optionalServices: ['battery_service']};
return navigator.bluetooth.requestDevice(options)
```

與先前一樣，取得裝置名稱並顯示電量後，請呼叫 `playbulbCandle.getBatteryLevel`，將此項目插入 `app.js`。

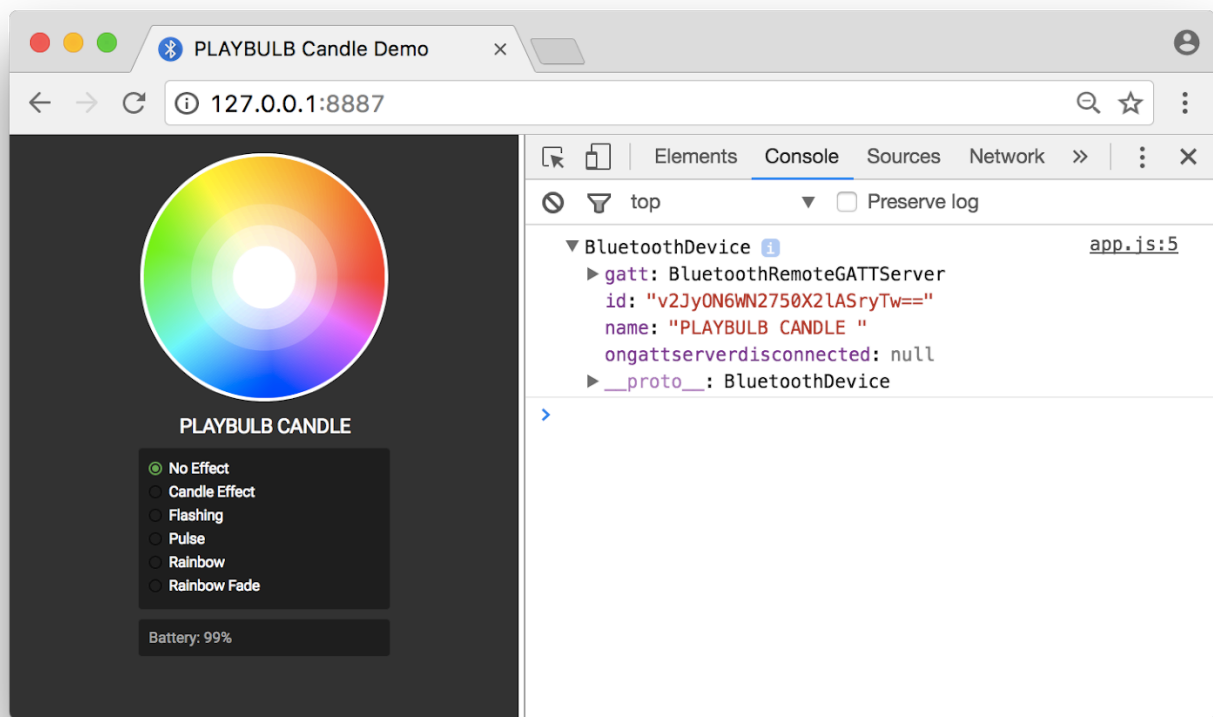
app.js

```
document.querySelector('#connect').addEventListener('click', event => {
  playbulbCandle.connect()
    .then(() => {
      console.log(playbulbCandle.device);
      document.querySelector('#state').classList.remove('connecting');
      document.querySelector('#state').classList.add('connected');
      return playbulbCandle.getDeviceName().then(handleDeviceName)
        .then(() => playbulbCandle.getBatteryLevel().then(handleBatteryLevel));
    })
    .catch(error => {
      console.error('Argh!', error);
    });
});

function handleDeviceName(deviceName) {
  document.querySelector('#deviceName').value = deviceName;
}

function handleBatteryLevel(batteryLevel) {
  document.querySelector('#batteryLevel').textContent = batteryLevel + '%';
}
```

此時，請在網路瀏覽器中造訪您的網站 (按一下網路伺服器應用程式中醒目顯示的網路伺服器網址)，或直接重新整理現有頁面。按一下頁面上的「連線」按鈕，即可查看裝置名稱和電量。



下一步

- How can I change the color of this bulb? 這就是我來這裡的原因！

- 你就快要成功了，相信我...

常見問題

- [哪裡可以讓我進一步瞭解 JavaScript ES2015 Promise？](#)
 - [哪裡可以找到官方支援的藍牙 GATT 特徵清單？](#)
 - [是否有藍牙偵錯控制台？](#)
-

步驟 6 · 約 6 分鐘

6. 變更顏色

如要變更顏色，只要在宣傳為 `0xFF02` 的主要 GATT 服務中，對藍牙特徵 (`0xFFFC`) 寫入特定指令集即可。舉例來說，如要將 PLAYBULB Candle 設為紅色，您需要編寫等於 `[0x00, 255, 0, 0]` 的 8 位元不帶正負號整數陣列，其中 `0x00` 是白色飽和度，`255, 0, 0` 則分別是紅色、綠色和藍色值。

我們將使用 `characteristic.writeValue`，在 `PlaybulbCandle` 類別的新 `setColor` 公開方法中，將一些資料實際寫入藍牙特徵。承諾兌現時，我們也會傳回實際的紅色、綠色和藍色值，以便稍後在 `app.js` 中使用：

playbulbCandle.js

```
const CANDLE_COLOR_UUID = 0xFFFC;

...

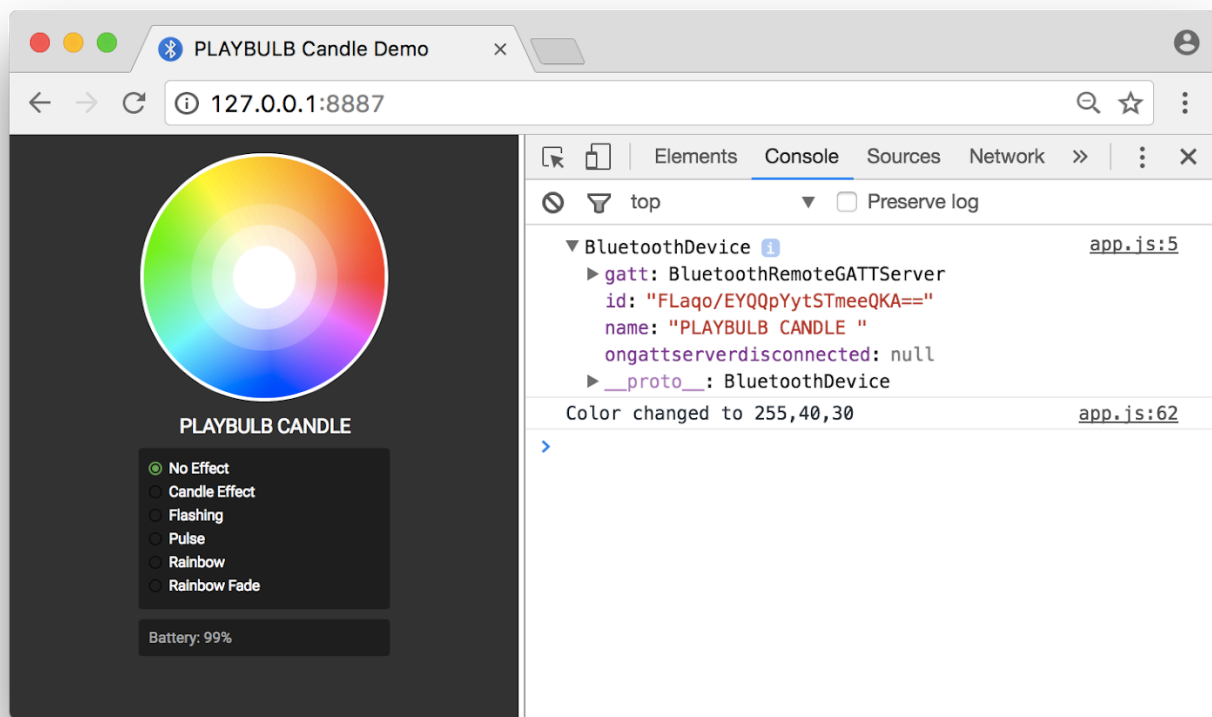
setColor(r, g, b) {
  let data = new Uint8Array([0x00, r, g, b]);
  return this.device.gatt.getPrimaryService(CANDLE_SERVICE_UUID)
    .then(service => service.getCharacteristic(CANDLE_COLOR_UUID))
    .then(characteristic => characteristic.writeValue(data))
    .then(() => [r,g,b]);
}
```

請更新 `app.js` 中的 `changeColor` 函式，在勾選「No Effect」單選按鈕時呼叫 `playbulbCandle.setColor`。使用者點選顏色挑選器畫布時，系統已設定全域 `r, g, b` 顏色變數。

app.js

```
function changeColor() {
  var effect = document.querySelector('[name="effectSwitch"]:checked').id;
  if (effect === 'noEffect') {
    playbulbCandle.setColor(r, g, b).then(onColorChanged);
  }
}
```

此時，請在網路瀏覽器中造訪您的網站 (按一下網路伺服器應用程式中醒目顯示的網路伺服器網址)，或直接重新整理現有頁面。按一下頁面上的「連線」按鈕，然後按一下顏色挑選器，即可隨意變更 PLAYBULB Candle 的顏色。



更多蠟燭效果

如果你點過蠟燭，就會知道燭光並非靜止不動，幸運的是，在廣告宣傳為 `0xFF02` 的主要 GATT 服務中，還有另一個藍牙特徵 (`0xFFFFB`)，可供使用者設定蠟燭效果。

如要為執行個體設定「蠟燭效果」，請編寫 `[0x00, r, g, b, 0x04, 0x00, 0x01, 0x00]`。您也可以使用 `[0x00, r, g, b, 0x00, 0x00, 0x1F, 0x00]` 設定「閃爍效果」。

我們將 `setCandleEffectColor` 和 `setFlashingColor` 方法新增至 `PlaybulbCandle` 類別。

playbulbCandle.js

```

const CANDLE_EFFECT_UUID = 0xFFFB;

...

setCandleEffectColor(r, g, b) {
  let data = new Uint8Array([0x00, r, g, b, 0x04, 0x00, 0x01, 0x00]);
  return this.device.gatt.getPrimaryService(CANDLE_SERVICE_UUID)
    .then(service => service.getCharacteristic(CANDLE_EFFECT_UUID))
    .then(characteristic => characteristic.writeValue(data))
    .then(() => [r,g,b]);
}

setFlashColor(r, g, b) {
  let data = new Uint8Array([0x00, r, g, b, 0x00, 0x00, 0x1F, 0x00]);
  return this.device.gatt.getPrimaryService(CANDLE_SERVICE_UUID)
    .then(service => service.getCharacteristic(CANDLE_EFFECT_UUID))
    .then(characteristic => characteristic.writeValue(data))
    .then(() => [r,g,b]);
}

```

接著，請更新 `app.js` 中的 `changeColor` 函式，在勾選「燭光效果」單選按鈕時呼叫 `playbulbCandle.setCandleEffectColor`，並在勾選「閃爍」單選按鈕時呼叫 `playbulbCandle.setFlashingColor`。這次我們將使用 `switch`，可以嗎？

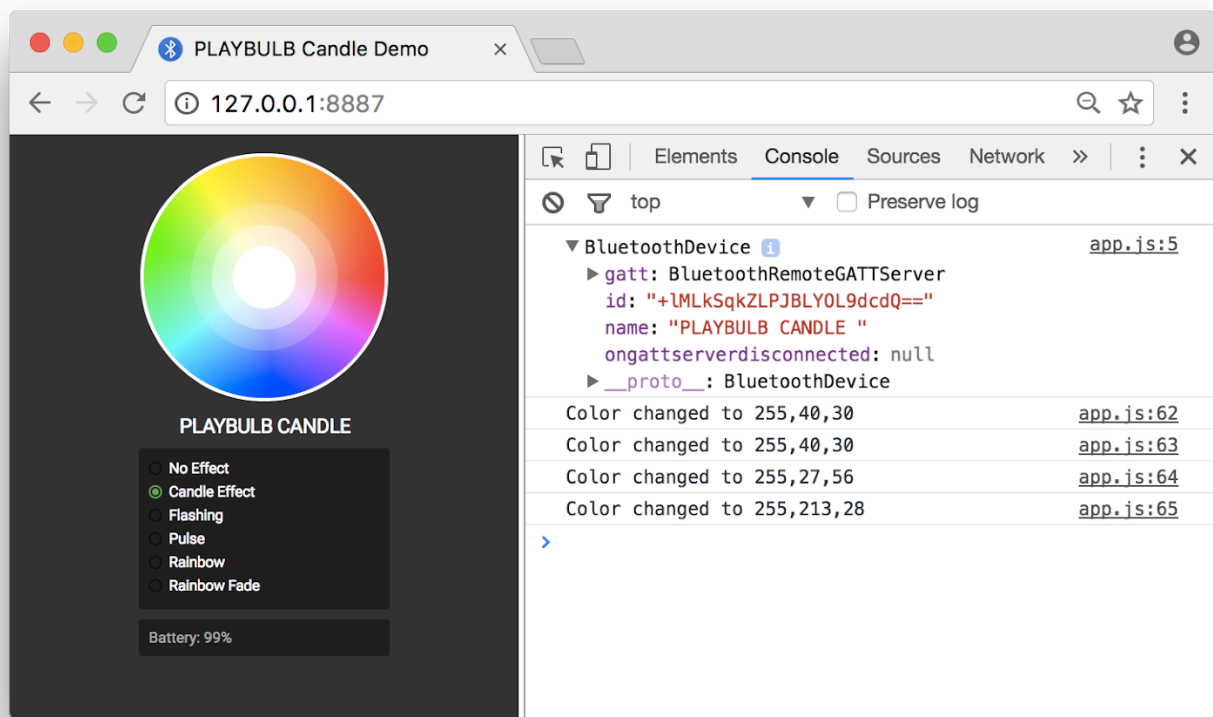
app.js

```

function changeColor() {
  var effect = document.querySelector('[name="effectSwitch"]:checked').id;
  switch(effect) {
    case 'noEffect':
      playbulbCandle.setColor(r, g, b).then(onColorChanged);
      break;
    case 'candleEffect':
      playbulbCandle.setCandleEffectColor(r, g, b).then(onColorChanged);
      break;
    case 'flashing':
      playbulbCandle.setFlashingColor(r, g, b).then(onColorChanged);
      break;
  }
}

```

此時，請在網路瀏覽器中造訪您的網站（按一下網路伺服器應用程式中醒目顯示的網路伺服器網址），或直接重新整理現有頁面。按一下頁面上的「連線」按鈕，即可使用蠟燭和閃爍效果。



下一步

- 這樣就完成了嗎？3 個蠟燭效果不佳？這就是我來這裡的原因嗎？
 - 還有更多，但這次要靠你自己了。
-

7. 再努力一下

我們到了！你可能以為快要結束了，但應用程式還沒完成。我們來看看您是否真的瞭解在本程式碼研究室中複製及貼上的內容。現在請自行完成下列步驟，讓應用程式更加出色。

新增缺少的特效

以下是缺少效果的資料：

- 脈衝：`[0x00, r, g, b, 0x01, 0x00, 0x09, 0x00]` (你可能想在那裡調整 `r, g, b` 值)
- 彩虹：`[0x01, 0x00, 0x00, 0x00, 0x02, 0x00, 0x01, 0x00]` (癲癇患者可能不適合使用這個主題)
- 彩虹漸層：`[0x01, 0x00, 0x00, 0x00, 0x03, 0x00, 0x26, 0x00]`

這基本上是指在 `PlaybulbCandle` 類別中新增 `setPulseColor`、`setRainbow` 和 `setRainbowFade` 方法，並在 `changeColor` 中呼叫這些方法。

修正「沒有效果」

如您所見，「無效果」選項不會重設任何進行中的效果，這點雖然微不足道，但仍值得注意。接著就來解決這個問題。在 `setColor` 方法中，您需要先透過新的類別變數 `_isEffectSet` 檢查效果是否正在進行，如果是，請先關閉效果，再使用下列資料設定新顏色：`[0x00, r, g, b, 0x05, 0x00, 0x01, 0x00]`。 `true`

輸入裝置名稱

這題很簡單！如要編寫自訂裝置名稱，只要寫入先前的藍牙裝置名稱特徵即可。建議使用 `TextEncoder` `encode` 方法取得包含裝置名稱的 `Uint8Array`。

接著，我會在 `document.querySelector('#deviceName')` 中新增「輸入」 `eventListener`，並呼叫 `playbulbCandle.setDeviceName` 來簡化作業。

我個人將其命名為「PLAY💡 CANDLE!」。

請注意，您隨時可以前往 <https://googlecode labs.github.io/candle-bluetooth/index.html> 查看這個應用程式的最終版本。

步驟 8 · 約 0 分鐘

8. 就是這麼簡單！

您已經瞭解的內容

- 如何在 JavaScript 中與附近的藍牙裝置互動
- 如何使用 ES2015 類別、箭頭函式、對應和 Promise

後續步驟

- 進一步瞭解 [Web Bluetooth API](#)
 - 瀏覽官方 [Web Bluetooth 範例](#)和[示範](#)
 - 看看[飛天臭臉貓](#)
-